

Inter-Die Swizzling优化

前提 融合算子从两层IR获得哪些信息

为了让训练与推理两类负载在晶圆级芯片上收敛为同一种表征，需要设计一种中间表示来表征跨Die编排。§ 4.1 已表明，该表征需要承载两类事实：其一，并行切分在分片状态发生变化之处所诱导的集合通信，包括其类型、数据量与参与者集合；其二，每个逻辑并行组所占据的子阵列，以及由该嵌入所导出的形状、拓扑闭合性与等效带宽。

我们进一步将IR拆分为两层。分层的IR在编译器设计中很常见 **引用占位：MLIR**，但此处层边界的划分依据并非抽象层次的高低，而是搜索的先后顺序。跨 die 部署本身是一个有序的三层嵌套搜索：并行切分先于位置映射确定，位置映射又先于 swizzle 模式的选择。为此，我们令 IR 的层边界与决策边界重合。IR-0 在切分确定之后固化，其所描述的集合通信类型与数据量不依赖于 die 的物理位置；IR-1 在位置映射确定之后固化，其所注入的嵌入信息不依赖于后续 swizzle 的选择。由此，每一层的搜索都可以共享上一层固化的IR：每层边界锁定一组已决定的参数，改动只需自该边界起重新计算，其上游结果可被缓存复用。在复杂的搜索空间中，这一性质直接决定了编译期搜索的可行性。

IR-0 描述与物理位置无关的逻辑计算通信图。其表示沿用张量分片的标准形式 **引用占位：GSPMD, DTensor**：设备集合由一个多轴的 device mesh 给出，每个轴对应一个并行维度；每个张量携带一个分片标注，记录其各逻辑维在 mesh 各轴上的分布，以及是否处于尚未归约的 partial 状态。在此表示之下，相邻算子之间分片状态的差量可以导出集合通信：例如，partial 转为 replicated 对应 AllReduce。因此，各并行维度得以在同一表示下统一：各类切分的差别仅在于分片标注绑定到 device mesh 的哪一根轴上，其所诱导的集合通信由上述差量规则一并给出。

IR-1 将上述逻辑图嵌入物理 die 阵列，为每个并行组注入一份嵌入描述。该描述由成员(rank)集合、逻辑形状、放置方案与互连结构四者唯一确定，其内容以组内成员对为索引展开：`hop_distance` 记录每一对成员之间的跳数，`pair_routes` 记录其确定性路由路径，二者刻画该组在阵列上的几何形态；`resource_incidence` 记录该路径所占用的物理资源（端口资源、D2D link资源等），`resource_capacity` 给出各资源的容量，二者构成代价计算的依据；`transit_dies` 则标出位于路径之上、承担转发却并不属于该组；`canonical_profiles` 中记录了每个并行组中相邻成员对之间的初始等效带宽。

相邻成员对之间的初始等效带宽，指考虑组间争用的情况下，本组中相邻成员对的路由路径所能支配的物理带宽份额，该带宽仅由映射决定，不考虑组内的实际流量模式。

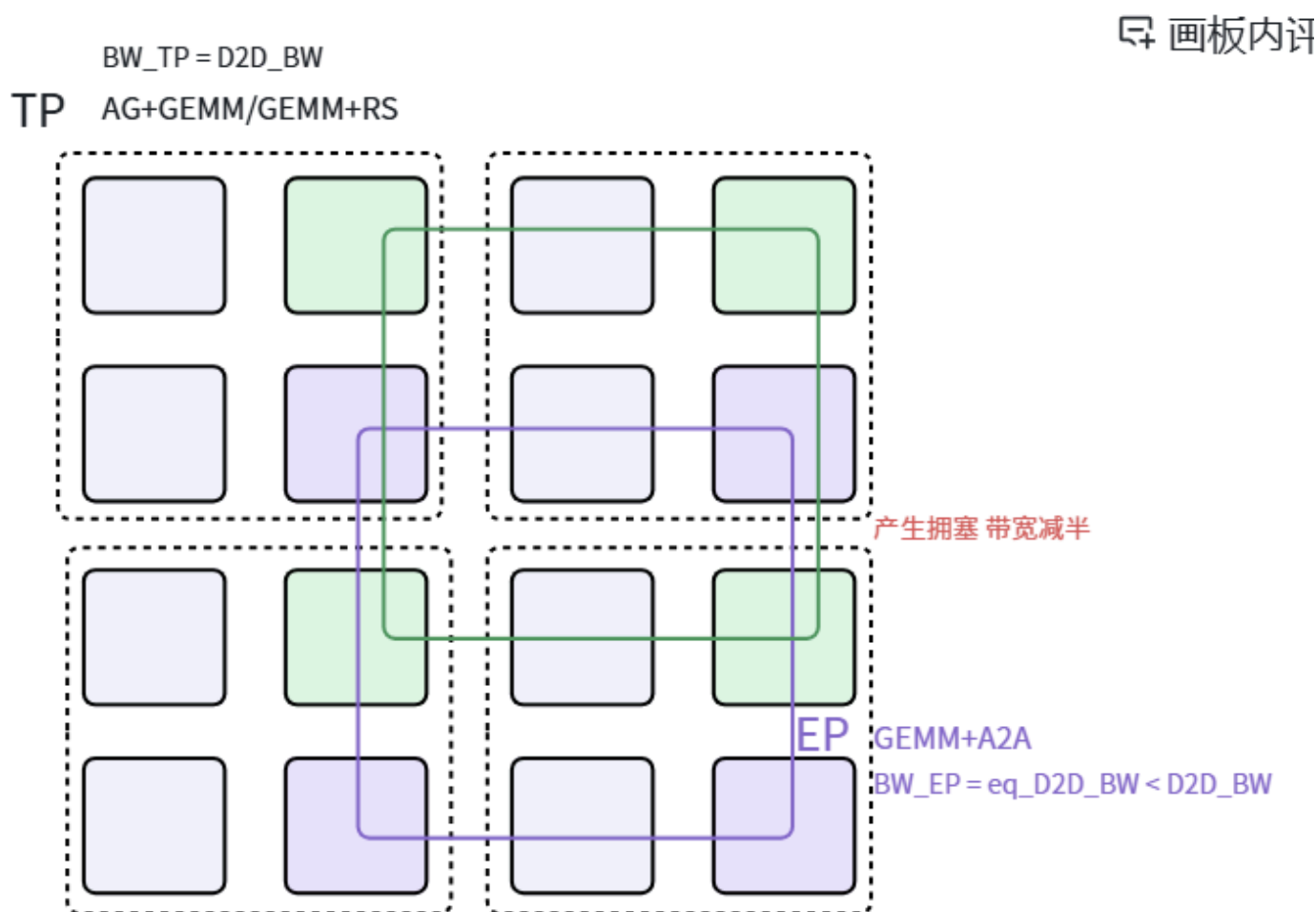
组间争用的范围是可判定的。在 SPMD 执行之下，阵列上同类的各并行组于同一时刻发起同一集合通信；记与组 g 同时活跃的同类组集合为 $P(g)$ （含 g 自身）。对其中任一组 g' ，其逻辑相邻成员对的路由所覆盖的全部资源构成该组的资源足迹 $fp(g')$ ，由 `resource_incidence` 逐对展开并取并即得。于是资源 r 上的争用组数与本组可支配容量分别为

$$K[r] = |\{g' \in P(g) : r \in \text{fp}(g')\}|, \quad C_g[r] = \frac{C[r]}{K[r]}$$

其中 $C[r]$ 由 `resource_capacity` 给出。相邻成员对 (a,b) 的流量须依次穿过其路径 $R(a,b)$ 上的每一个资源，故其等效带宽由其中最受限者决定：

$$B_{\text{eq}}(a, b) = \min_{r \in R(a, b)} C_g[r] = \min_{r \in R(a, b)} \frac{C[r]}{K[r]}$$

如图 X 所示，张量并行组占据连续的 2×2 区块，其相邻成员即物理近邻，各组流量互不越界，故每条链路上 $K[r]=1$ ，等效带宽即为标称 D2D 带宽；专家并行组则由各张量并行组中相同相对位置的 die 构成，其相邻成员相隔两跳，路径须借道中间 die，并与相邻专家并行组的路径共享链路，使瓶颈链路上 $K[r]=2$ ，等效带宽降至标称值的一半。二者位于同一块阵列、承受同样的并发，其差异仅来自嵌入方式。



融合算子在两层IR中逐级产生：IR-0 依据分片状态的变迁识别出 GEMM 与相邻集合通信，通过判断数学等价性来确认融合合法性，将合法的融合方案登记为融合候选。IR-1 具备了放置、分块与缓冲信息，可以判定融合候选的可执行性。

经两次校验成立的融合算子会同时携带两层信息：(1) IR-0提供的算子自身规模信息：GEMM 的形状与计算量、集合通信的类型与字节数，二者共同决定该算子的计算通信比。(2) IR-1提供的物理阵列信息：所占子阵列的形状与拓扑闭合性、成员对之间的跳数与路由、相邻成员对间的等效带宽等。

因此，融合算子已同时具备计算规模、通信量与物理阵列三类信息，这些信息为融合算子内部计算和通信的交错方式选择提供基础。

优化思路

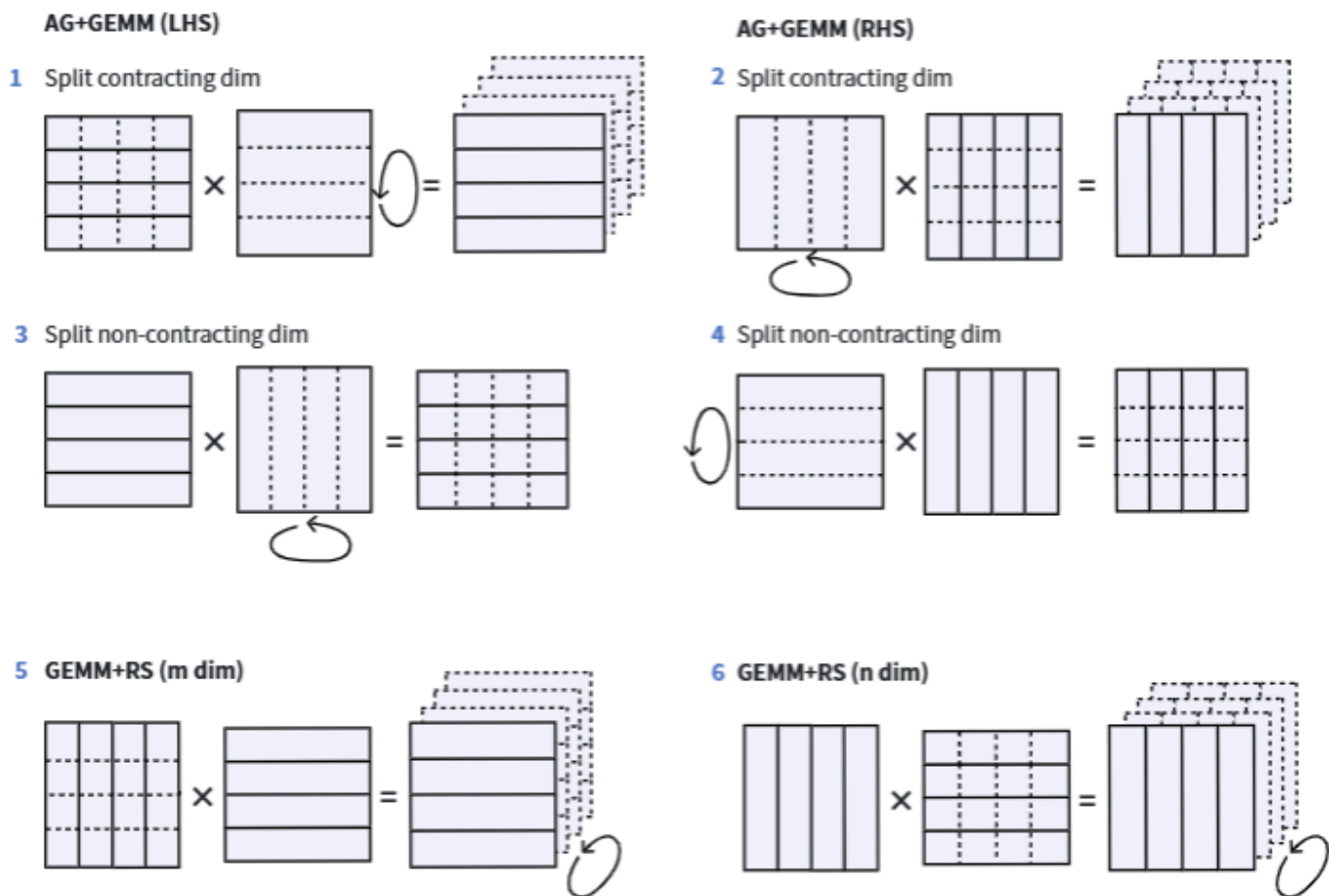
GEMM+Collective算子在获得作用的GEMM大小与Mesh信息之后，如何进行这一块的Swizzling选择与匹配

候选Swizzle模式

GEMM+RS/AG+GEMM/GEMM+AR

AG+GEMM、GEMM+RS 与 GEMM+AR 三类融合算子共享同一结构：三者的集合通信都属于非个性化通信每个字节需抵达或来自多个成员，其内容在成员间共享。**引用占位 Grama et al., § 4.2** 它们的差别仅在通信位于算子的前侧或后侧、以及归约是否伴随再分发。它们因此共享同一候选集。这类算子有两个候选。

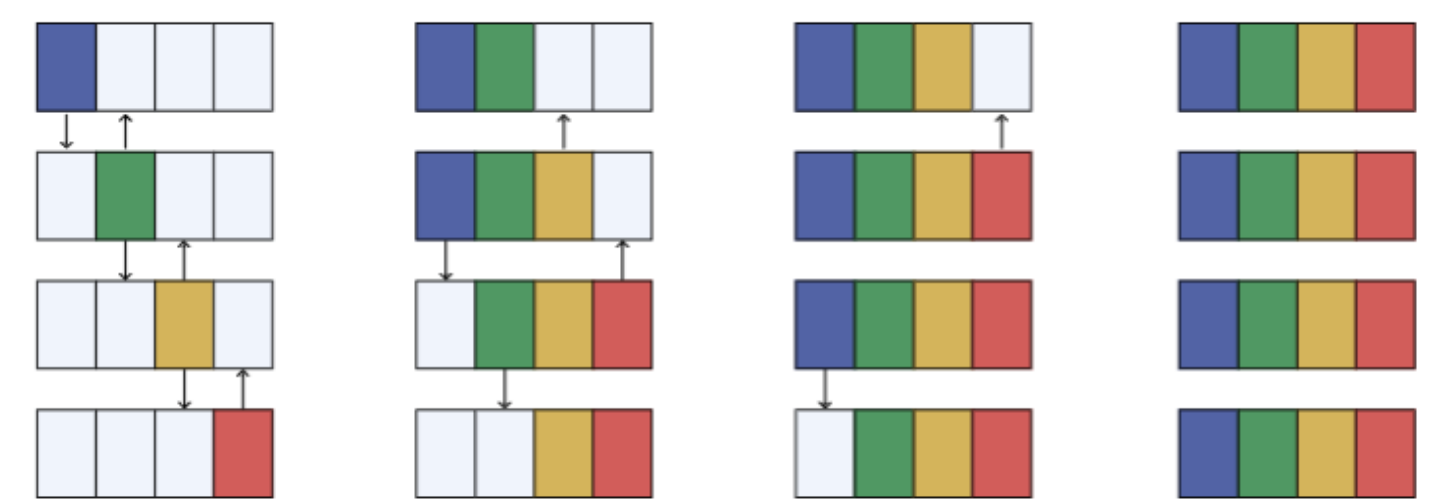
候选一：1D Ring Swizzling。 其切分取一维：沿 GEMM 的 M 或 N 维将张量划分为与成员数相等的若干块，按 LHS 被切轴落在非收缩维、收缩维还是 batch 维，可进一步分为三种情形，分别对应 AllGather 前置、ReduceScatter 后置与二者退化为无通信的情况。其**通信算法**取环上的循环移位（collective permute）：每一步各成员将持有的块沿环推进一跳，步后恰好完成一次 AllGather 或 ReduceScatter。其**重排**由此确定性地导出——AllGather+GEMM 的每一步在收到的块上执行一次子 GEMM 并写入结果的对应切片；GEMM+RS 的每一步则把 `looped_operand` 用作累加器，且令起始索引与成员编号刻意错开，以保证步后每个 scatter 切片恰好停在其目的成员上（**引用占位：Google OD, ASPLOS'23**）。该模式有两个来源不同的子变体：一是直接在二维 mesh 上嵌入环、由片上互连原生支持的形态，二是先由一维张量并行产出 AG+GEMM / GEMM+RS 对，再套用上述重叠切分法的形态。其优点在于依赖结构在环上构成全序，控制简单、流水稳定、无需全局同步；其代价在于，环是一维结构，而 mesh 上每个 die 有四个方向的出口端口，单向环在任一时刻只使用其中一个方向，其余端口全程空转——按 § 4.2 的资源模型，这一损失可直接在 `resource_incidence` 上量化为端口利用率的上界。此外，环的可用性依赖于 § 4.1 所给出的**拓扑闭合性**：所占子阵列上须存在一条哈密顿回路，否则环的相邻成员之间将出现多跳绕行，逻辑相邻与物理相邻的偏离会直接反映为等效带宽的下降。



候选二：2D Row-Column Swizzling。 其切分取二维：将 GEMM 的两个非收缩维分别映射到子阵列的行与列，成员按 的二维网格持有子块，并按哪一个操作数驻留不动，分为输出驻留、输入驻留与权重驻留三种变体（引用占位：**MeshSlice, ISCA'25**）。其通信算法取行、列两族环上的并行集合通信：行内环与列内环同时推进，且各自被进一步切成多个小规模 AllGather，以便与计算逐段重叠。其重排规则相应简化——由于每个子块只需其行伙伴与列伙伴的对应片段，收缩维索引对齐即视为依赖满足，计算可在任一方向的片段到达后立即启动，无须等待整轮通信完成。相对于一维环，其优势有二：其一，行、列两族环同时占用互相正交的端口方向，聚合带宽的利用率成倍提高；其二，成员间的最大距离由 降至 ，在晶圆级 达 量级时这一差别是决定性的。其代价则在于控制复杂度更高，且性能对子阵列的长宽比敏感：当 时两族环的长度不等，短边一侧先行完成而长边一侧成为瓶颈；同时行、列两族流量在同一物理资源上可能相互争用，其程度已由 § 4.2 的等效带宽给出——那里的例子表明，同一并行组内行向与列向的等效带宽可以相差一倍，这一不对称性正是本候选在具体放置下是否划算的判据来源。

两个候选并非优劣关系。一维环以牺牲带宽利用换取控制的简单与流水的稳定，二维网格以更高的控制复杂度换取带宽与距离上的优势，孰优取决于融合算子的形状（）、子阵列的形状（）与等效带宽三者的具体取值——而这三项恰好都已由 § 4.2 的融合算子随身携带。§ 4.3.2 将说明如何据此在二者之间作出选择。

这里只考虑MeshSlice中OS的做法，且由于MeshSlice是在Torus上做，而晶圆级芯片一维不成环，所以一维上进行AG/AR/RS的做法为：1d bidirectional line AG/AR/RS



Dispatch+GEMM/GEMM+Combine

Dispatch+GroupGEMM 与 GroupGEMM+Combine 属于跨轴重分片类融合算子。专家并行下，token 在进入专家层之前按序列位置分片于各成员，之后须按其路由到的专家重新分布——分片所依附的轴由此从序列位置换成专家归属，Combine 则执行其逆过程。与轴内重分片不同，这里没有任何一个成员的数据是被多个成员共享的：每个 token 只送往其选中的 个专家，每个字节的目的地是确定且少数的。按集合通信的经典分类，这是**个性化通信**（引用占位：Grama et al., § 4.5）。

这一差异使前述三个成分中的两个失效，而第三个得以保留。**重排规则保留**：生产者侧先算依赖已就绪者、消费者侧先算目的地最远者，这两条是拓扑无关的方向性公理，与通信是否个性化无关。**通信算法失效**：环之所以在轴内重分片类上成立，是因为一次面向邻居的传输同时服务了下游全部成员，中继复用率为；个性化通信的复用率恒为，代价退化为体量与平均距离之积，而环把 的二维结构嵌入一维，平均距离由 抬升至。经典结果给出的两种代价（引用占位：Grama et al., § 4.5.1, § 4.5.2）其带宽项相差；晶圆级 达 量级时这是二十倍以上的差距。**切分粒度的静态性亦失效**：GroupGEMM 每个专家所收 token 数由运行时路由决定， 维不定长，切分边界无法如轴内重分片类那样在编译期算死。

因此本族需要独立的候选模式，而候选一（1D Ring Swizzling）不在其中——这不是经验取舍，而是上述个性化性质的直接推论。

通信算法采用二阶段行列 A2A：第一阶段，各成员将其 token 按目的地所在的**列**打包，在所属行内完成一次 A2A；第二阶段，各成员将收到的包按目的地所在的**行**在所属列内再作一次 A2A。该算法是经典结果（引用占位：Grama et al., § 4.5.2），本文沿用而非重新设计。值得指出的是它与直接 A2A 的关系：在 XY 维序路由下， 的包本就在 处转向，二阶段算法只是把这一转向从硬件提升到软件，并在转向处对同去一列的包作聚合。二者的链路负载因而相同，消息数则由 降至，单包尺寸放大 倍——晶圆级上每次传输都需占用一个 DTE 描述符与接收侧的控制资源，消息数是真实成本。

但对本文而言，二阶段更关键的价值不在通信代价而在**结构**：直接 A2A 是一次无内部次序的突发，无处插入计算；二阶段使转向成员成为一个天然的流水级边界，通信被切成两级，chunk 得以在两级之间滚动推进。**这才是 A2A 能够被编排、进而与计算重叠的前提**。同时，该算法与轴内重分片类的候选二共享同一结构——行、列两族并行推进——候选空间因此在两族算子上保持形式一致，唯一的自由维度仍是切分维数。

切分沿用 COMET 的方向性选择（引用占位：COMET）：Dispatch+GroupGEMM 侧沿 token 维（）切分，chunk 的自然边界取（源成员所属到达层，目的专家）二元组；GroupGEMM+Combine 侧沿 维切分，首批 列算完即可归约并发出，从而使输出的产生与回传同时进行。

重排由前两者导出，但其排序键在 mesh 上比在交换机上更丰富。COMET 在全互连域内只能区分"本地"与"远端"两档，因为源成员的身份不携带代价信息；在二阶段 mesh 上，到达时刻由源成员所需经历的阶段数与跳数共同决定，形成分层结构：本地或（仅需一个阶段）两阶段（按跳数递增）GroupGEMM 的 tile 次序即按此层序推进。该排序键的全部要素——跳数、路由、阶段划分——恰由 § 4.2 的 IR-1 随算子携带。且由于各源成员归属哪一层完全由放置决定、与运行时路由无关，接收缓冲可按层**静态分桶**，token 到达即落入正确的桶内，tile 边界沿桶顺序推进，无需任何为地址连续而作的搬运——与前述"重排是零数据搬运的纯索引变换"这一约束一致。

本族算子的动态性来自路由：哪些 token 去哪个专家在运行时才确定，故各 chunk 的字节数无法在编译期算死。但需要区分的是，**动态的只是流量的体量，其形状是静态的**。专家的放置、每对成员间的路由与所经资源、等效带宽、阶段划分、tile 的层序规则，均由 § 4.1 与 § 4.2 在编译期确定；落到 § 4.2 的表示上，`rank_programs` 完全静态，运行时仅需绑定 `chunk_slices` 中的 `bytes` 一项。若进一步采用容量因子对每专家的 token 数施加上限，则连该项亦退化为静态上限，整个 `FusionPlan` 完全在编译期确定。因此 § 4.3.2 的匹配算法无需为本族单独设计运行时自适应机制，只需在代价模型中将各成员对的流量体量视为带分布的参数而非定值。

负载的动态均衡不是本文的重点，但二阶段算法的流量结构提示了一个放置层面的可利用性质：第一阶段的流量按**目的列**聚合，故热点专家若集中于同一列，将使该列在第一阶段承担远超均值的注入量。将热点专家在列间分散，即可在不改变任何编排的前提下均衡两个阶段的负载。此类放置目标与 § 4.1 的子阵列选择同属一个层次，本文将其列为可与放置联合优化的方向，不在本节展开。

搜索算法

1. 总纲：这不是一个搜索器，而是一个四级判定管线

§ 4.3.1 的统一分解（编排 = 切分 × 通信算法 → 重排，重排为导出量）已把搜索空间从三维压到二维，且每族算子的候选只有 1-2 个 pattern、每个 pattern 内的切分自由度只有几十个点。因此匹配可以做到**近似穷举，无迭代搜索**。这一点应在小节开头与末尾各说一次——它是 § 4.3.1 那个分解式的最终回报，也是对 ASPLOS 读者的直接说服力（编译时间不构成负担）。

级	做什么	代价
0	拓扑闭合性谓词 (哈密顿回路 / 完整矩形)	O(1) 布尔
1	闭式区间判据定 pattern，输出长度 1-2 的短名单	O(1) 查表
2	穷举（切分维度 × chunk 数），用 B_{eq} 算 T	几十次评估
3	部署难度 + 动态性 → 微调 chunk 数 → 决断	每候选一次

2. 目标函数

主目标为融合算子的估计执行时间：

$$T(\sigma) \; ; = \; T_{\text{pro}}(\sigma) \; ; + \; \max\big(T_{\text{comp}}(\sigma), T_{\text{comm}}(\sigma)\big) \; ; + \; T_{\text{epi}}(\sigma)$$

T_{pro} 为首个 chunk 无法被掩盖的部分（gather 型为首块通信，scatter 型为首块计算）， T_{epi} 为末块的对偶，中间项为稳态流水。**分块数只影响首尾两项与效率，不影响中间项——**这是选 chunk 数的全部张力所在。

选 T 而非"重叠率"作主目标：T 可加、可跨算子比较、可直接汇入端到端估计；重叠率只在算子内有意义。

辅助指标（用于并列时决断）：

指标	定义	是否含估计误差
暴露通信比 ε	$\max(0, T_{\text{comm}} - T_{\text{comp}}) / T$	有
端口方向利用率 u	同时活跃的出口方向数 / 4	无（从 plan 直接数）
控制复杂度 c	同步点数 + 每成员动作数 + 在飞 chunk 数	无（从 rank_programs 直接数）

后两项无估计误差，这一性质在第 3 级用于打破由估计误差造成的并列。

3. 第 0 级：可行性剪枝（前置于任何代价评估）

用 IR-1 的拓扑闭合性作纯布尔判定：

- **1D 环形编排**：子阵列上须存在哈密顿回路
- **2D 正交环编排**：子阵列须为完整矩形，且行/列方向闭合
- **二阶段行列 A2A**：子阵列须为完整矩形

两条谓词的失败后果性质不同：前者候选消失，后者降级为非聚合 XY A2A（见 1190 行）。前置的理由是**剪枝不消耗代价模型**，而代价模型是全流程唯一昂贵的部分。

4. 第 1 级：闭式判据初选 pattern（轴内重分片类）

沿用已有区间判据 ($q = d \cdot \text{comp} / \text{bw}$ ，见 1044-1048 行)：

区域	条件	优者	幅度
① 双 compute-bound	$m \geq \frac{q}{2}xy$	1D 环形（控制简单、流水稳）	≈ 1
② 仅 Ring comm-bound	$\frac{m}{xy} < \frac{q}{2} \leq \min(\frac{m}{y}, \frac{m}{x})$	2D 正交环	$1.5-3\times$
③ 双 comm-bound	$\min(\frac{m}{y}, \frac{m}{x}) < \frac{q}{2}$	2D 正交环	$\sim \sqrt{P}$

三点必须写清：

1. **这是查表不是搜索**。q 的两个因子一个来自 IR-0 (comp)、一个来自 IR-1 (bw)，融合算子自带，O(1) 得出。此处直接回收 § 4.2 末段埋的伏笔。
2. **必须代入真实的 (x,y) 而非假定方阵**。§ 4.1 已证子阵列形状随负载/放置/故障变化，x≠y 的长条会显著压缩 2D 的优势——**区间边界随运行时子阵列移动，这正是"必须匹配而非静态选定"的理由**，是本小节存在的合法性来源。
3. **边界附近不裁**。q/2 落在阈值误差带内时，两候选一并送入第 2 级。本级输出是**有序短名单**（长度 1-2），非唯一解。

跨轴重分片类跳过本级（候选唯一）。训练/推理先验（训练→2D、推理→1D，见 1038 行）在此作为短名单的排序初始化，**而非替代判据**。

4. 第 2 级：穷举切分

pattern	切分维度	粒度	空间大小
1D 环形	M 或 N	chunk 数 $\in$ 约数集	~ 10
2D 正交环	维度到行列的指派 $\times$ stationary 变体 (OS/IS/WS)	chunk 数	~ 30
二阶段 A2A	固定 (dispatch 沿 M, combine 沿 N)	两阶段各自 chunk 数	~ 10

chunk 数的最优点由三条**静态可得**的约束夹出：

- 上界：SRAM 容量与在飞 chunk 上限（IR-1 的 FusionPlanValidator 已给出）
- 上界：每次传输占用 DTE 描述符与接收侧控制资源 $\rightarrow$ chunk 太小则消息开销主导
- 下界：低于片上高效 tile 尺寸时计算效率崩塌（见第 6 点的 η ）

5. 关于第二级等效带宽：本文候选集下退化为恒等（重要，需改写 § 4.2 措辞）

命题：本文候选集中每条逻辑边被展开为一条 P2P 流；候选的可行性谓词恰好保证这些流在链路与端口上两两不交——1D 环形编排要求哈密顿回路，故环上逻辑相邻即物理相邻，P 条边落在 P 条互不相同的链路上；2D 正交环的行族与列族分别占用正交的端口方向。因此**组内自争用在链路与端口层面恒不发生**， $B_{\sigma} \equiv B_{\text{eq}}$ 。

剩余的只有端点资源（NoC / HBM）。**⚠ 待确认的硬件前提：**若 NoC 带宽高于该 die 四方向端口带宽之和，端点侧永不成为瓶颈，该项亦消失。此不等式属 § 2.2 待补的量化数值；若一时无数据，正文中标为一条显式假设。

连锁效应（比想象中好，不要简单删掉两级论述）： $B_{\sigma} \equiv B_{\text{eq}}$ 意味着 § 4.2 那个 $\leq$ 号收紧为等号， B_{eq} 从上界升格为**可达值**。于是：

1. 第 2 级的剪枝从"保守剪枝"变成**精确比较**；
2. 匹配算法的误差源收敛到**唯一一处：片上计算时间**，使 § 4.3.2 论述干净很多；
3. 候选集的质量得到侧面印证——可行性谓词不只保证"能跑"，还顺带保证"无自争用"，这是可明写的非平凡性质。

§ 4.2 的处理方式：保留两级结构作为**框架**（容纳未来引入的、流量会重叠的候选），但明确指出在**本文候选集下第二级退化为恒等**，实现中略去。既诚实，又不浪费已写好的形式化。

⚠ 唯一例外：多播树 / 归约树折叠一旦启用，上述命题失效——多播树的定义即多个目的地共享路径前缀，共享路径正是流量重叠；归约树同理。建议先按**不采用**来写；若后来决定保留树折叠，那部分自带一个局部的分支带宽分析即可，不必为它维持通用的两级框架。

（附：据此撤回此前"端点资源在匹配算法内部被用上、§ 4.2 措辞需放宽"的建议。若端点不构成瓶颈，该层资源仍只在端到端评估（PD 分离的 KV handoff）时需要，§ 4.2 原措辞不必改。）

4. 第3级：部署难度 + 动态性 → 微调 → 决断

本级不复算带宽（见第5点），三项输入为：

- **动态性**：MoE 流量体量是分布而非定值 → 按期望值与尾部（如 p95）双点评估 chunk 数，避免只优化均值；
- **部署难度**：控制复杂度 c 与 SRAM 压力，均可从 `rank_programs` 直接数出，无估计误差；
- **微调切分**：以上两项改变最优 chunk 数，对每个存活候选在 chunk 数上重跑一次一维扫描（ ~ 10 点）。

决断规则：

1. 存活候选的 $[T_{\text{lo}}, T_{\text{hi}}]$ 区间**不相交** → 按 T 定；
2. **相交** → 按 (u, c) 字典序决断（先取端口方向利用率高者，再取控制复杂度低者）。用无误差量打破由估计误差造成的并列，是自洽的；
3. **仍并列** → 不强行决断，把并列候选一并写入 `FusionPlan` 的 profile 映射，交由 §5 的片上编排最终解决。§4.2 的 `plan` 本就是 `profile_id → FusionPlan` 的映射而非单值，输出小候选集与语义完全相容。

4. 片上计算时间怎么估（全节最硬的一处）

存在**真实的循环依赖**：§4.3 要 T_{comp} 才能定跨 die 编排，而 T_{comp} 取决于 §5 的片上切分与映射，§5 又需要 §4.3 的 chunk 形状作输入。三条出路，建议同时用后两条：

- **(a) 解析建模片上**——把 die 内当作 $\text{cores} \times \text{NoC}$ 的小型同类问题递归求解。优雅但风险高：片上还有 SRAM 层次、bank 冲突、控制核开销，建模误差未必小于经验法，且会把 §5 的复杂度提前引入 §4.3。**不建议。**
- **(b) 标定的效率曲线**——离线在一组 tile 形状上微基准测量该 die 的实际算力效率 η_{tile} ，则 $T_{\text{comp}} = \text{FLOPs} / (\text{peak} \cdot \eta)$ 。 η 把片上切分与映射的效果**吸收为一条经验曲线**，无需显式建模；它只依赖 die 架构、与本次编排无关，可随硬件 profile 缓存。它同时提供第2级所需的 **chunk 尺寸下界**（ η 崩塌处即最小高效 chunk）。
- **(c) 关键论证：候选之间比较时，绝对误差大部分可约去。**所有候选执行**同一个 GEMM**，总 FLOPs 相同；候选间 T_{comp} 的差异**只来自两处**——分块粒度对 η 的影响，以及负载是否均衡。因此匹配算法需要的是**相对准确**的计算时间模型，而非绝对准确的。 η 曲线哪怕整体偏移 20%，只要其**形状**（随 tile 尺寸的相对变化）可信，候选排序就不变。

在此之上加一层保险：区间传播。用 η 的置信带给出 $[T_{\text{lo}}, T_{\text{hi}}]$ ，按第6点的决断规则处理。这样 §4.3 与 §5 之间形成**干净的接口而非循环**：

§4.3.2 的输出不必是单一方案，而可以是一个**带约束的小候选集**——每个候选附带它对片上编排提出的要求（chunk 形状、最小高效 tile、在飞缓冲），§5 在解决片上切分时顺带选定其一。

这不是权宜：它承认跨 die 与片上两层决策不可完全解耦，并给出一个**不需要迭代**的解耦方式——上层传递约束与候选集，下层收敛。建议在 §4.3.2 明写，并在 §5 开头呼应。

5. 复杂度小结

每个融合算子：第 0 级 $O(1)$ 谓词，第 1 级 $O(1)$ 查表，第 2 级几十次代价评估，第 3 级对 1-2 个存活候选各一次。**全程无迭代搜索**，编译时间不构成问题。

6. 待拍板

- a. **\eta 曲线从哪来？** 若 § 7 评估平台是仿真器（mNPUSim?）可直接跑微基准；若是解析模型，\eta 需自建，第 7 点 (c) 的论证强度会下降。**这关系到本节能写多硬。（想一下\eta曲线的构建）**
- b. **NoC 带宽 vs 四方向端口带宽之和**（第 5 点的 ⚠️）——决定第二级能否彻底删去。
- c. **区间传播是否写进正文？** 严谨但多花半页，且需给出 \eta 置信带来源。篇幅紧则退化为"点估计 + 敏感性分析放 § 7"。
- d. **第 3 级 tie-break 用 (u,c) 字典序 vs 加权和。** 字典序无需标定权重，但可能在 u 上过度偏好 2D。